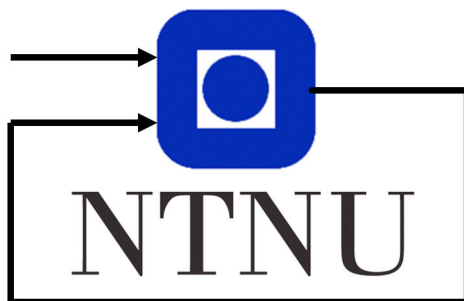


TTK4135 Helicopter Lab Report

Oliver Sanchez

April 9th, 2025



Department of Engineering Cybernetics

Contents

1	10.2 - Optimal Control of Pitch/Travel without Feedback	1
1.1	The continuous model	1
1.2	The discretized model	2
1.3	The open loop optimization problem	3
1.4	The weights of the optimization problem	4
1.5	Experimental results	6
1.6	MATLAB and Simulink	8
2	10.3 - Optimal Control of Pitch/Travel with Feedback (LQ)	12
2.1	LQ controller	12
2.2	Model Predictive Control	13
2.3	Experimental results	14
2.4	MATLAB and Simulink	17
3	10.4 - Optimal Control of Pitch/Travel and Elevation with Feedback	21
3.1	The continuous model	21
3.2	The discretized model	22
3.3	Experimental results	23
3.4	Decoupled model	23
3.5	MATLAB and Simulink	28
3.6	Optional exercise	31
	3.6.1 Option A	31
	3.6.2 Option B	34
	3.6.3 Option C	36

1 10.2 - Optimal Control of Pitch/Travel without Feedback

1.1 The continuous model

Answer 10.2.1.1

State and Input Definitions: We define the state vector and the control input as

$$x = \begin{pmatrix} \lambda \\ r \\ p \\ \dot{p} \end{pmatrix}, \quad u = p_c,$$

where:

- λ is the travel angle,
- r is the travel rate ($\dot{\lambda}$),
- p is the pitch angle,
- $\dot{p}$ is the pitch rate,
- p_c is the pitch control reference.

Using the init09 configuration file given for helicopter 9 we know the following parameters; K_1 , K_2 , K_{pp} , and K_{pd} that characterize the physical dynamics and the inner-loop controller gains.

Continuous-Time Dynamics: The system dynamics are modeled by the differential equation

$$\dot{x} = A_c x + B_c u,$$

where the matrices are given by

$$A_c = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -K_2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -K_1 K_{pp} & -K_1 K_{pd} \end{pmatrix}, \quad B_c = \begin{pmatrix} 0 \\ 0 \\ 0 \\ K_1 K_{pp} \end{pmatrix}.$$

This formulation captures the travel and pitch dynamics of the helicopter. Here, the parameter K_2 appears in the coupling between the travel rate and the pitch angle. The fourth row of A_c models the pitch acceleration, where the term $-K_1 K_{pp}$ represents the proportional control action and $-K_1 K_{pd}$ models the derivative action. The input matrix B_c shows that the control input p_c affects only the pitch acceleration with a gain of $K_1 K_{pp}$. This model not only describes the physical dynamics of the helicopter but also reflects the inner-loop control strategy, as illustrated in Figure 7 of the lab task handout (not figure 7 in my lab report!).

1.2 The discretized model

Answer 10.2.1.2. Remember to document the calculations.

Starting Point: We start with the continuous-time model

$$\dot{x}(t) = A_c x(t) + B_c u(t),$$

where

$$A_c = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -K_2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -K_1 K_{pp} & -K_1 K_{pd} \end{pmatrix}, \quad B_c = \begin{pmatrix} 0 \\ 0 \\ 0 \\ K_1 K_{pp} \end{pmatrix},$$

and the state $x = [\lambda; r; p; \dot{p}]$ with input $u = p_c$.

Using the forward Euler Approximation: Using a sampling time $T = 0.25$ seconds (as set by `dt = 0.25`), we approximate the derivative via

$$\dot{x}(t) \approx \frac{x(t+T) - x(t)}{T}.$$

At discrete time $t = kT$ (denoting $x_k = x(kT)$ and $u_k = u(kT)$), this results in

$$\frac{x_{k+1} - x_k}{T} \approx A_c x_k + B_c u_k.$$

Discrete-Time Model: Multiplying both sides by T and rearranging, we obtain

$$x_{k+1} = x_k + T (A_c x_k + B_c u_k).$$

This expression is equivalent to the standard discrete-time state-space form:

$$x_{k+1} = (I + T A_c) x_k + T B_c u_k.$$

Defining the discrete system matrices as

$$A = I + T A_c, \quad B = T B_c,$$

and substituting $T = 0.25$, the model becomes

$$x_{k+1} = A x_k + B u_k,$$

1.3 The open loop optimization problem

Decision Variables: First, we bundle the state and control variables into one large optimization vector:

$$z = \begin{pmatrix} x_1^\top & x_2^\top & \cdots & x_N^\top & u_0^\top & u_1^\top & \cdots & u_{N-1}^\top \end{pmatrix}^\top.$$

Cost Function: The objective is to minimize a quadratic cost that penalizes the deviation of the travel angle from a desired final value, λ_f , and the use of control effort:

$$\min_z J(z) = \sum_{i=0}^{N-1} \left[(\lambda_{i+1} - \lambda_f)^2 + q (p_{c,i})^2 \right],$$

where $q \geq 0$ is a weighting parameter that balances the emphasis between tracking accuracy and control effort.

Constraints: The optimization is subject to:

1. **Dynamic Constraints:** These enforce the system's discrete-time dynamics for $k = 0, \dots, N-1$:

$$x_{k+1} = A x_k + B u_k.$$

2. **Initial and Terminal Conditions:** The initial state is given by

$$x_0 = \begin{pmatrix} \lambda_0 \\ 0 \\ 0 \\ 0 \end{pmatrix},$$

and the terminal state is often prescribed as

$$x_N = \begin{pmatrix} \lambda_f \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

3. **Inequality Constraints:** For example, a physical bound on the pitch angle is given by

$$|p_k| \leq \frac{60\pi}{360}, \quad k = 1, \dots, N.$$

Note that similar bounds on the pitch reference p_c are not necessarily required because the system dynamics ensure the relationship between p and p_c .

Quadratic Program Formulation: Finally, by suitably stacking all the decision variables z , the optimization can be cast as a standard quadratic programming (QP) problem:

$$\min_z \frac{1}{2} z^\top H z + f^\top z$$

subject to linear equality constraints $A_{eq} z = b_{eq}$ (from the dynamics and boundary conditions) and inequality constraints $A_{ineq} z \leq b_{ineq}$ (from the pitch bounds).

1.4 The weights of the optimization problem

q=0.12:

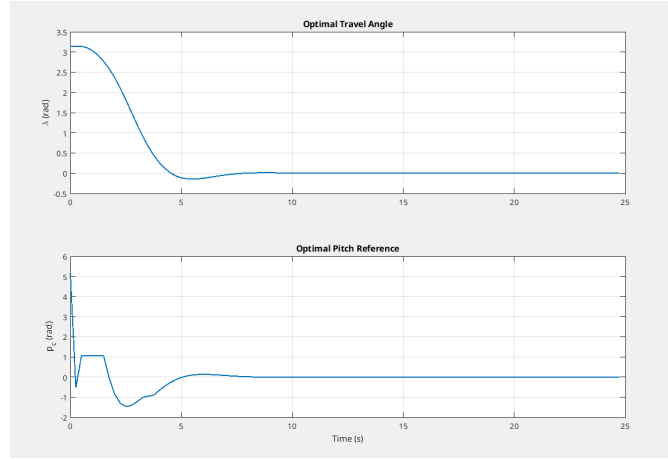


Figure 1: optimal travel angle and pitch reference for q=0.12

q=1.2:

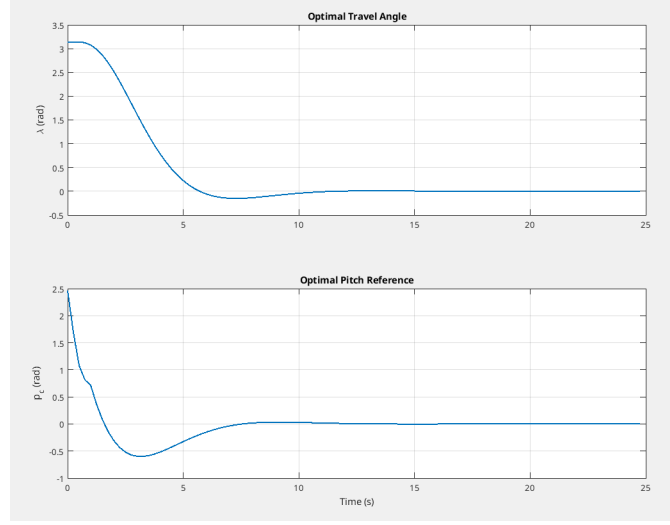


Figure 2: optimal travel angle and pitch reference for $q=1.2$

q=12:

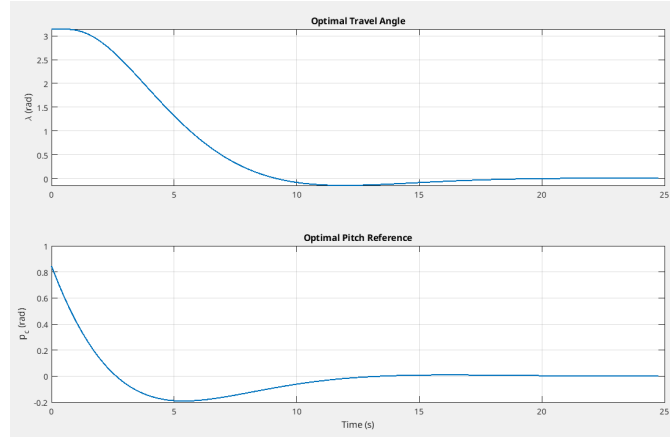


Figure 3: optimal travel angle and pitch reference for $q=12$

For the daring tasks:

- **Penalizing λ at each timestep vs. terminal penalty:** Penalizing $(\lambda_{i+1} - \lambda_f)^2$ at all time points will drive λ toward λ_f . But, since $\lambda_0 \neq \lambda_f$, the early values of λ_i would result in a large cost in the function. This approach can still work, but one it could be better to penalize primarily the final deviation $(\lambda_N - \lambda_f)^2$ (i.e., a “terminal cost”) if the main goal is to ensure the final state is correct without over-penalizing the initial transient.

- **Scaling vs. the control term $qp_{c,i}^2$:** The relative magnitude of $(\lambda_{i+1} - \lambda_f)^2$ and $qp_{c,i}^2$ is determined by how large q is. A larger q would mean the optimizer limits the pitch reference p_c significantly to effort spent on control, this could possibly slow down the convergence to λ_f . A smaller q puts more emphasis on driving λ to λ_f quickly, and hence it would tolerate a larger $|p_{c,i}|$.
- **Alternative formulations:** One can still drive λ to λ_f by using, for instance,

$$\sum_{i=0}^{N-1} (\lambda_N - \lambda_f)^2 + \alpha \sum_{i=0}^{N-1} p_{c,i}^2,$$

or by including a terminal state constraint $\lambda_N = \lambda_f$. These methods focus mainly on the final or terminal deviation in the penalty rather than on every timestep. But both strategies still ensure $\lambda \rightarrow \lambda_f$.

- **Dealing with $\lambda_f + 2\pi$ being physically the same angle:** The cost $(\lambda_{i+1} - \lambda_f)^2$ does not recognize that these two are in fact representing the same orientation; λ_f and $\lambda_f + 2\pi$. Using a modulo operation

$$\min_u \sum_{i=0}^{N-1} (\text{mod}(\lambda_{i+1} - \lambda_f, 2\pi))^2,$$

would capture equivalences of angles. This would likely introduce non-smooth behavior in the cost function and can create multiple local minima, which would complicate solution via gradient-based QP solvers. Practically, the helicopter has no wiring constraints, spinning multiple rotations to $\lambda_f + 2\pi$ is physically allowed, but the standard quadratic cost does not consider such equivalences.

1.5 Experimental results

I ran experiments with $q=1$ as specified in the lab instructions. This provided a quite good result in terms of smoothness, but I did not realize at the time that the tracking was considerably off. Furthermore, visual inspection showed that the helicopter was just traveling in circles, around and around. This continuous traveling in circles was due to the not being any feedback loop so the inputs were just predefined to be 0 and yet the helicopter would just keep going in circles, never knowing how close or far it was from the correct track.

Optimal angle and pitch with $q=1$:

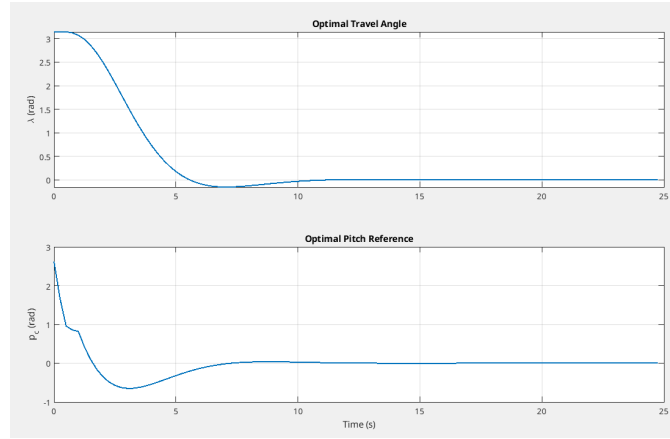


Figure 4: My test figure

During the helicopter tests I logged the state of the helicopter (all six of them), the inputted pitch reference, and the optimal values of all six helicopter states.

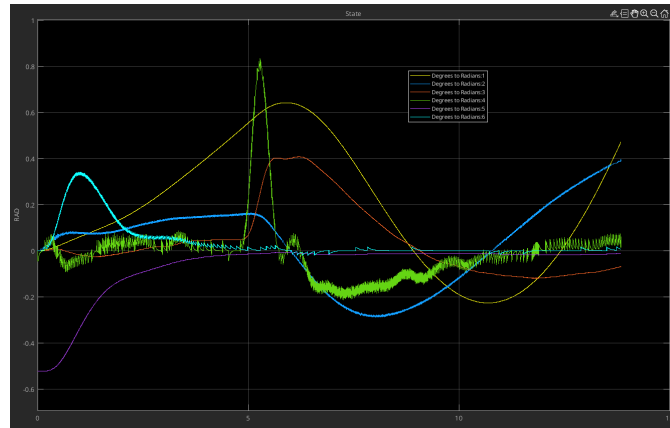


Figure 5: Helicopter state, the legend is organized in the same order as the state equations. I.e. λ , λ_{dot} , p , p_{dot} , e , e_{dot}

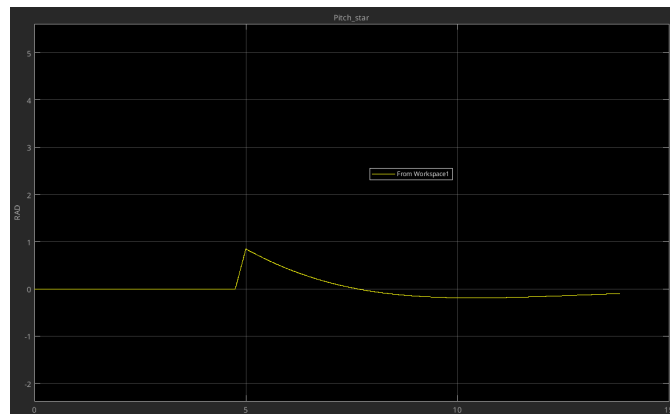


Figure 6: Pitch reference (padded with 0s first 5 seconds as instructed in lab task.)

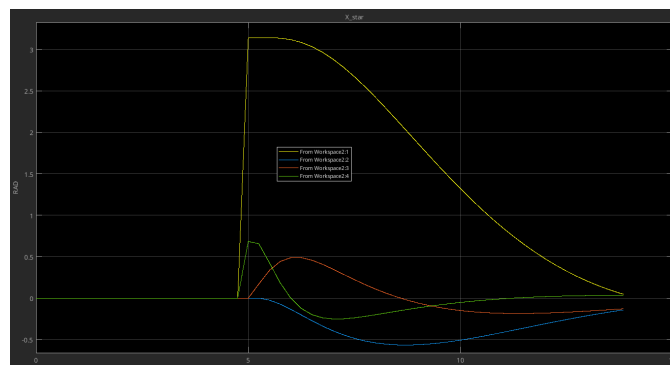


Figure 7: Optimal trajectory of all helicopter states, legend being in same order as the state graph (padded with 0s first 5 seconds as instructed in lab task.)

1.6 MATLAB and Simulink

Code and diagrams go here (only lines that you've written yourself). Give some explanation of the code if necessary.

So I did not use any of the templates given, so I will include the code I have written for producing the time series, and then also include the simulink setup I had for task 10.3 (i just bypassed the feedback by inputting a dummy K_{opt} which is just the same size as K but with a lot of zeros.

```
clear;
clc;

init09;
```

```

A_c = [0 1 0 0;
        0 0 -K_2 0;
        0 0 0 1;
        0 0 -K_1*K_pp -K_1*K_pd];

B_c = [0;
        0;
        0;
        K_1*K_pp];

dt = 0.25;
A = eye(4) + dt * A_c;
B = dt * B_c;

N = 100;
nx = 4;
nu = 1;

x0 = [pi; 0; 0; 0];
xf = [0; 0; 0; 0];

q = 1;
Q = diag([1, 0, 0, 0]);
R = q;

z_len = N*nx + N*nu;

H = zeros(z_len, z_len);
f = zeros(z_len, 1);

for k = 1:N

    H((k-1)*nx+1:(k-1)*nx+nx, (k-1)*nx+1:(k-1)*nx+nx) = Q;
end

for k = 1:N
    idx = N*nx + (k-1)*nu + 1;
    H(idx:idx+nu-1, idx:idx+nu-1) = R;
end

Aeq = zeros(N*nx, z_len);
beq = zeros(N*nx, 1);

for k = 1:N
    xk_idx = (k-1)*nx+1;
    uk_idx = N*nx + (k-1)*nu + 1;

    if k == 1
        Aeq(xk_idx:xk_idx+nx-1, 1:nx) = eye(nx);
        Aeq(xk_idx:xk_idx+nx-1, uk_idx:uk_idx+nu-1) = -B;
    end
end

```

```

        beq(xk_idx:xk_idx+nx-1) = A * x0;
    else
        Aeq(xk_idx:xk_idx+nx-1, (k-2)*nx+1:(k-2)*nx+nx) = -A;
        Aeq(xk_idx:xk_idx+nx-1, xk_idx:xk_idx+nx-1) = eye(nx);
        Aeq(xk_idx:xk_idx+nx-1, uk_idx:uk_idx+nu-1) = -B;
    end
end

pitch_max = deg2rad(60);
Aineq = zeros(N*2, z_len);
bineq = zeros(N*2, 1);

for k = 1:N
    xk_idx = (k-1)*nx + 3;

    Aineq(2*k-1, xk_idx) = 1;
    bineq(2*k-1) = pitch_max;

    Aineq(2*k, xk_idx) = -1;
    bineq(2*k) = pitch_max;
end

opts = optimoptions('quadprog', 'Display', 'off');
[z_opt, ~, exitflag] = quadprog(H, f, Aineq, bineq, Aeq, beq, [], [], [],
    opts);

if exitflag ~= 1
    error('Optimization failed!');
end

x_opt = reshape(z_opt(1:nx*N), nx, N)';
u_opt = z_opt(nx*N+1:end);

time = (0:N-1) * dt;

figure;
subplot(2,1,1);
plot(time, x_opt(:,1), 'LineWidth', 1.5); grid on;
ylabel('\lambda (rad)');
title('Optimal Travel Angle');

subplot(2,1,2);
plot(time, u_opt, 'LineWidth', 1.5); grid on;
ylabel('p_c (rad)');
xlabel('Time (s)');
title('Optimal Pitch Reference');

%data we need to send to simulink
padding_time = 5;
pad_steps = round(padding_time / dt);

```

```

u_full = [zeros(pad_steps,1); u_opt; zeros(pad_steps,1)];

x_pad_start = repmat(x0', pad_steps, 1);
x_pad_end = repmat(xf', pad_steps, 1);

x_full = [zeros(pad_steps,4); x_opt; zeros(pad_steps,4)];
t_full = (0:length(u_full)-1)' * dt;

pc_timeseries = timeseries(u_full, t_full);

u_star_timeseries = timeseries(u_full, t_full);
x_star_timeseries = timeseries(x_full, t_full);
save('pc_timeseries.mat', 'pc_timeseries');

disp(x_star_timeseries);
disp(u_star_timeseries);
save('timeseries.mat', 'x_star_timeseries','u_star_timeseries');

load('lqr_gain.mat');
load('timeseries.mat');
disp(K)
disp(K_opt)
size(K)

```

Listing 1: Optimal control in MATLAB

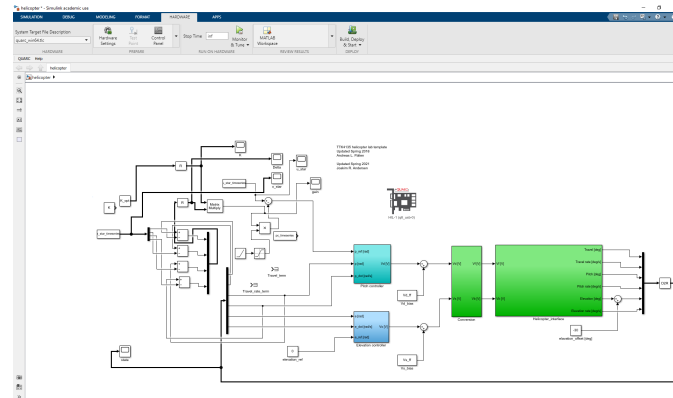


Figure 8: Simulink diagram of 10.2. The K_{opt} is a dummy vector that is empty so no gain is applied. I change only this connection to quickly switch between experiments in 10.2 and 10.3.

2 10.3 - Optimal Control of Pitch/Travel with Feedback (LQ)

2.1 LQ controller

2.1 LQ Controller:

- **Concept:** A Linear Quadratic (LQ) controller in discrete time solves

$$\min_{\{\Delta u_i\}} \sum_{i=0}^{\infty} \left[\Delta x_{i+1}^{\top} Q \Delta x_{i+1} + \Delta u_i^{\top} R \Delta u_i \right]$$

subject to

$$\Delta x_{i+1} = A \Delta x_i + B \Delta u_i,$$

where $\Delta x_i = x_i - x_i^*$ and $\Delta u_i = u_i - u_i^*$.

- **What do Q and R really do in effect:**
 - $Q \geq 0$ penalizes state deviations Δx . Larger entries in Q (try to) force the state to track the desired trajectory more aggressively.
 - $R > 0$ penalizes the manipulated variable deviations Δu . Larger entries in R (try to) reduce control effort at the expense of allowing bigger state deviations. (these two are the tunable matrices to change the behavior of your device)
- **Choice of weights:** One typically uses diagonal matrices for Q and R . The diagonal entries reflect how severely each state and each control channel is penalized. A good heuristic is to start with moderate values for Q and R , then tune them to meet performance requirements (this is what took me the most time in this report, its a large part of control engineering, but also just understanding them, getting a feel for how they affect things is something I think is key)
- **Implementation:** In MATLAB, one can compute the LQ gain matrix K using

$$K = \text{dlqr}(A, B, Q, R).$$

2.2 Model Predictive Control

2.2 (10.3.1.3) Model Predictive Control:

- **Strategy (A) – Re-optimize each time:** We go through each strategy and then its pros and cons in practice.
 - At each sampling instant, measure the new state $x(k)$.
 - Formulate a finite-horizon optimal control problem starting from $x(k)$.
 - Solve it (e.g., via QP) to get a new optimal input sequence u^* .
 - Apply only the first input $u^*(k)$, discard the rest, and repeat at the next sampling.

Advantages:

- Continuously adapts to changes in environment or errors in the modeling.
- Handles state/input constraints naturally at each step.

Disadvantages:

- Requires solving an optimization problem online at every step (computational load is often quite sensitive when on e.g. a light drone).
- **Strategy (B) – MPC to stabilize the initially predicted trajectory:**
 - We would start with an initial predicted trajectory x^* .
 - then use a constrained or unconstrained receding-horizon scheme around x^* (basically a “tracking” MPC).

Advantages/Disadvantages:

- Similar feedback benefits to strategy A, but designed specifically to stay near x^* .
- In the case that x^* no longer reflects the real system’s best course, strategy B may be suboptimal compared to re-optimizing from scratch in strat A.

Possible combination: A hybrid approach might re-derive a new x^* from time to time (like strat A does) while also using an MPC-based stabilizer (like in strat B) between the re-optimizations, this could balance the extra computation with the improved robustness.

2.3 Experimental results

Printouts of data from relevant experiments (plots). Discussion and analysis of the results. Answer 10.3.2.5 here.

In this experiment I got stuck early on. I derived all the equations and calculated the K matrix quite nice, the values seem a bit high but nothing that was too extreme I thought, as long as the delta would not explode. Now the issue I got was that the delta was usually not negligible, and multiplying by the K factors which were often in the range of 10-40 I got a very sensitive control loop with explosive reactions. This resulted in an unstable control that made the helicopter flip from side to side, which I could include a video because it was a bit extreme to watch (no equipment was damaged in any way though, I made sure to stop the experiments quickly once it got out of control). Here is some of the first experimental I got.

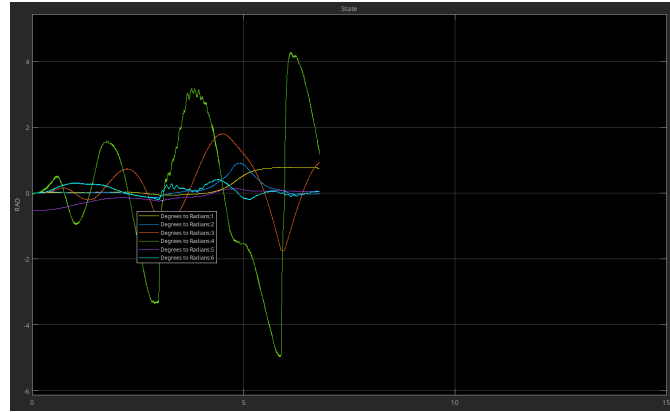


Figure 9: state of the helicopter, using the theoretically correct K values

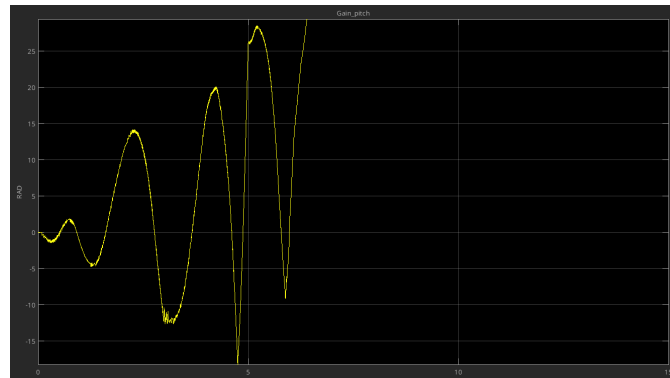


Figure 10: the gain of the pitch throughout

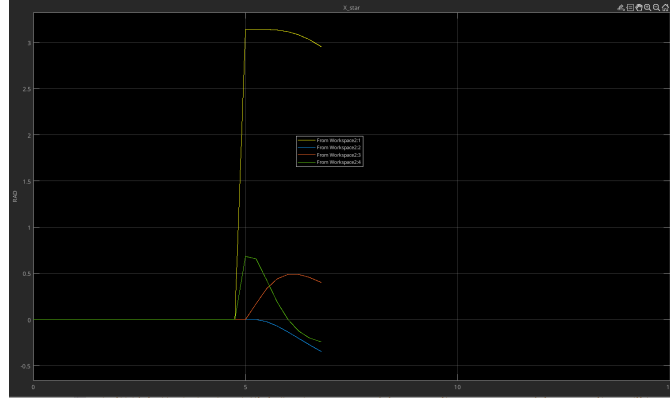


Figure 11: Here is the trajectory calculated and what is being used to calculate the delta (difference between this trajectory and the helicopter state)

It is obvious to see that the gain just grows out of control. Now the delay is still being used and this causes the delta to be quite large at 5 seconds as the trajectory is saying the lambda should go from 0 to pi. This however does not explain why the states are going out of control before those 5 seconds as at that time all values should be going to 0, which they were not (this does not include the elevation of course as we are only controlling the pitch by modeling the first four parameters).

I struggled a lot with this task and it did not matter how I tuned the Q and R matrices (controlling how much deltas are penalized in the lqr), the K matrix always got large, and the experiment would always be a failure. I went on to look for answers online, but did not find much of any help. I ended up analyzing the problem myself and decided to just hard code a division by 20 for the K matrix, forcing the gains to be much smaller and reasonable. This finally led to satisfactory performance. Here is the data from the experiments where I did this hacky solution:

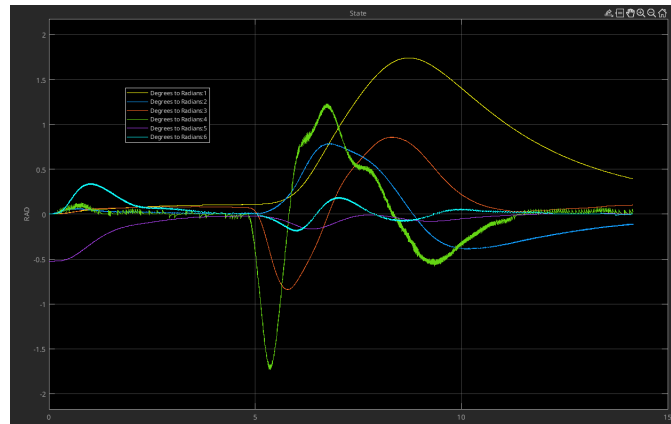


Figure 12: state of the helicopter, using the theoretical K divided by 20

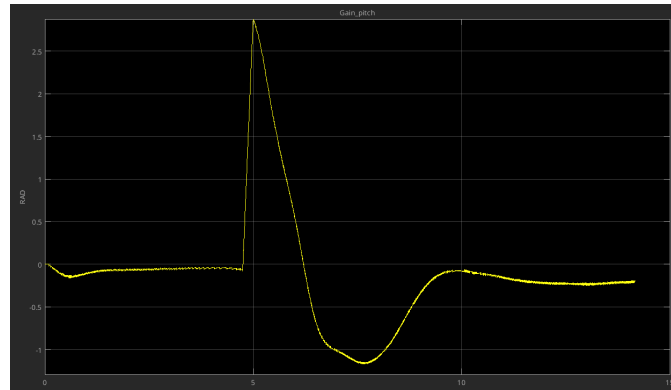


Figure 13: the gain of the pitch throughout

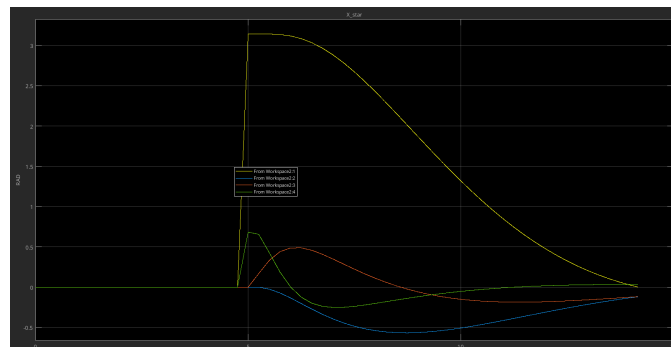


Figure 14: Here is the trajectory calculated and what is being used to calculate the delta (difference between this trajectory and the helicopter state)

We can see that the states are a lot more controlled in the first five

seconds and then change due to the delta increasing as the optimal trajectory suddenly changes reference values after five seconds. The gain can be seen to react strongly initially, to make up for this correction and then becoming negative when the state has been overcorrected. It then stabilizes nicely and the helicopter states go to zero, bringing the task to an end. The helicopter's overall trajectory was correct, going in a pi radians lambda change before stopping the travel motion and staying stable in the air.

2.4 MATLAB and Simulink

Code and diagrams go here

```
clear;
clc;

init09;

A_c = [0 1 0 0;
       0 0 -K_2 0;
       0 0 0 1;
       0 0 -K_1*K_pp -K_1*K_pd];

B_c = [0;
       0;
       0;
       K_1*K_pp];

dt = 0.25;
A = eye(4) + dt * A_c;
B = dt * B_c;

N = 100;
nx = 4;
nu = 1;

x0 = [pi; 0; 0; 0];
xf = [0; 0; 0; 0];

q = 1;
Q = diag([1, 0, 0, 0]);
R = q;

z_len = N*nx + N*nu;

H = zeros(z_len, z_len);
f = zeros(z_len, 1);

for k = 1:N
```

```

        H((k-1)*nx+1:(k-1)*nx+nx, (k-1)*nx+1:(k-1)*nx+nx) = Q;
    end

    for k = 1:N
        idx = N*nx + (k-1)*nu + 1;
        H(idx:idx+nu-1, idx:idx+nu-1) = R;
    end

    Aeq = zeros(N*nx, z_len);
    beq = zeros(N*nx, 1);

    for k = 1:N
        xk_idx = (k-1)*nx+1;
        uk_idx = N*nx + (k-1)*nu + 1;

        if k == 1
            Aeq(xk_idx:xk_idx+nx-1, 1:nx) = eye(nx);
            Aeq(xk_idx:xk_idx+nx-1, uk_idx:uk_idx+nu-1) = -B;
            beq(xk_idx:xk_idx+nx-1) = A * x0;
        else
            Aeq(xk_idx:xk_idx+nx-1, (k-2)*nx+1:(k-2)*nx+nx) = -A;
            Aeq(xk_idx:xk_idx+nx-1, xk_idx:xk_idx+nx-1) = eye(nx);
            Aeq(xk_idx:xk_idx+nx-1, uk_idx:uk_idx+nu-1) = -B;
        end
    end

    pitch_max = deg2rad(60);
    Aineq = zeros(N*2, z_len);
    bineq = zeros(N*2, 1);

    for k = 1:N
        xk_idx = (k-1)*nx + 3;

        Aineq(2*k-1, xk_idx) = 1;
        bineq(2*k-1) = pitch_max;

        Aineq(2*k, xk_idx) = -1;
        bineq(2*k) = pitch_max;
    end

    opts = optimoptions('quadprog', 'Display', 'off');
    [z_opt, ~, exitflag] = quadprog(H, f, Aineq, bineq, Aeq, beq, [], [], [],
        opts);

    if exitflag ~= 1
        error('optim didn't work:');
    end

    x_opt = reshape(z_opt(1:nx*N), nx, N)';

```

```

u_opt = z_opt(nx*N+1:end);

time = (0:N-1) * dt;

figure;
subplot(2,1,1);
plot(time, x_opt(:,1), 'LineWidth', 1.5); grid on;
ylabel('\lambda (rad)');
title('Optimal Travel Angle');

subplot(2,1,2);
plot(time, u_opt, 'LineWidth', 1.5); grid on;
ylabel('p_c (rad)');
xlabel('Time (s)');
title('Optimal Pitch Reference');

%data we need to send to simulink
padding_time = 5;
pad_steps = round(padding_time / dt);

u_full = [zeros(pad_steps,1); u_opt; zeros(pad_steps,1)];

x_pad_start = repmat(x0', pad_steps, 1);
x_pad_end = repmat(xf', pad_steps, 1);

x_full = [zeros(pad_steps,4); x_opt; zeros(pad_steps,4)];
t_full = (0:length(u_full)-1)' * dt;

pc_timeseries = timeseries(u_full, t_full);

u_star_timeseries = timeseries(u_full, t_full);
x_star_timeseries = timeseries(x_full, t_full);
save('pc_timeseries.mat', 'pc_timeseries');

disp(x_star_timeseries);
disp(u_star_timeseries);
save('timeseries.mat', 'x_star_timeseries', 'u_star_timeseries');

load('lqr_gain.mat');
load('timeseries.mat');
disp(K)
disp(K_opt)
size(K)

```

Listing 2: Optimal control in MATLAB

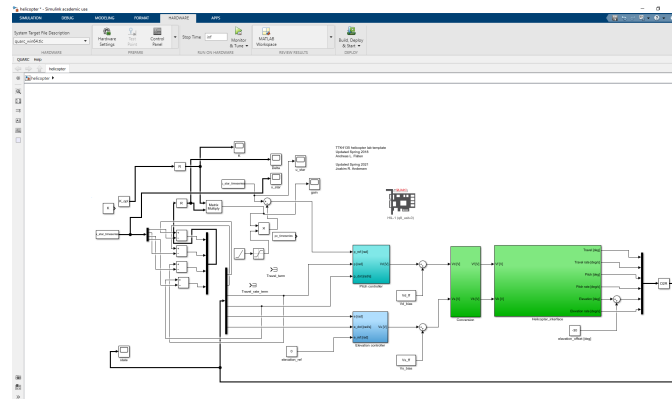


Figure 15: Simulink diagram of 10.3. The K_{opt} , dummy vector, is changed to the calculated K values to produce the intended LQR setup for this experiment.

3 10.4 - Optimal Control of Pitch/Travel and Elevation with Feedback

3.1 The continuous model

State and Input Definitions: We extend the state vector to include the elevation dynamics. Define

$$x = \begin{pmatrix} \lambda \\ r \\ p \\ \dot{p} \\ e \\ \dot{e} \end{pmatrix} \quad \text{and} \quad u = \begin{pmatrix} p_c \\ e_c \end{pmatrix},$$

where:

- λ : travel angle,
- r : travel rate ($\dot{\lambda}$),
- p : pitch angle,
- $\dot{p}$: pitch rate,
- e : elevation angle,
- $\dot{e}$: elevation rate,
- p_c : pitch setpoint (manipulated variable for pitch control),
- e_c : elevation setpoint (manipulated variable for elevation control).

The physical parameters and controller gains determine the constants:

$$K_1, \quad K_2, \quad K_{pp}, \quad K_{pd}, \quad K_3, \quad K_{ep}, \quad K_{ed}.$$

Continuous Dynamics: The continuous-time state-space model is given by

$$\dot{x} = A_c x + B_c u,$$

with

$$A_c = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & -K_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -K_1 K_{pp} & -K_1 K_{pd} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -K_3 K_{ep} & -K_3 K_{ed} \end{pmatrix}, \quad B_c = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ K_1 K_{pp} & 0 \\ 0 & 0 \\ 0 & K_3 K_{ep} \end{pmatrix}.$$

Discussion: This model now reflects the extended dynamics of the helicopter including both travel-pitch and elevation. The first four rows of A_c and B_c capture the travel and pitch dynamics (identical to the model without elevation), while the last two rows introduce the elevation dynamics. In particular, the term $-K_3K_{ep}$ in A_c models how the elevation dynamics are affected by the elevation force and the damping is provided by $-K_3K_{ed}$. The inputs p_c and e_c are the setpoints for the inner-loop controllers managing the pitch and elevation, respectively.

3.2 The discretized model

Starting Point: We start from the continuous-time model

$$\dot{x}(t) = A_c x(t) + B_c u(t),$$

with

$$x(t) = \begin{pmatrix} \lambda(t) \\ r(t) \\ p(t) \\ \dot{p}(t) \\ e(t) \\ \dot{e}(t) \end{pmatrix}, \quad u(t) = \begin{pmatrix} p_c(t) \\ e_c(t) \end{pmatrix},$$

and the matrices

$$A_c = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & -K_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -K_1K_{pp} & -K_1K_{pd} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -K_3K_{ep} & -K_3K_{ed} \end{pmatrix}, \quad B_c = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ K_1K_{pp} & 0 \\ 0 & 0 \\ 0 & K_3K_{ep} \end{pmatrix}.$$

Using the forward Euler approximation: With a sampling time of $T = 0.25$ seconds (as defined by $\text{dt} = 0.25$), the derivative is approximated by

$$\dot{x}(t) \approx \frac{x(t+T) - x(t)}{T}.$$

At discrete time instants $t = kT$ (with $k = 0, 1, \dots, N-1$), we have

$$\frac{x_{k+1} - x_k}{T} \approx A_c x_k + B_c u_k.$$

Deriving the Discrete Model: Multiplying by T and rearranging yields

$$x_{k+1} = x_k + T(A_c x_k + B_c u_k).$$

This can be expressed in standard discrete-time form:

$$x_{k+1} = \left(I + T A_c \right) x_k + T B_c u_k.$$

Defining

$$A = I + T A_c, \quad B = T B_c,$$

and noting that I is the 6×6 identity matrix, the resulting model is

$$x_{k+1} = A x_k + B u_k.$$

This discrete formulation is needed to optimize numerically with `fmincon` and for implementing the feedback control in Simulink.

3.3 Experimental results

3.4 Decoupled model

Observation of Decoupling: In the derived model, the first four states

$$\begin{pmatrix} \lambda \\ r \\ p \\ \dot{p} \end{pmatrix}$$

(i.e., the travel and pitch dynamics) are independent of the last two states

$$\begin{pmatrix} e \\ \dot{e} \end{pmatrix}$$

(i.e., the elevation dynamics). This decoupling arises because the model equations have been linearized and the coupling terms (often involving nonlinear functions such as $\cos(p)$) have been neglected.

Why our controller doesn't just crash: There are two primary reasons why the decoupled-model based controller works in practice:

1. **Operating Point Approximation:** In the nominal operating region, where our pitch p and elevation e do not deviate that much from their equilibrium, the nonlinear coupling terms remain nearly constant (often close to 1, since $\cos(0)=1$). Thus, the decoupled linear model provides a good approximation of the actual dynamics.
2. **Robust Feedback Compensation:** The feedback controller is designed to stabilize deviations from the optimal trajectory. Even if the model neglects certain nonlinear couplings, the feedback loop can compensate for unmodeled dynamics, provided that the deviations are not too big.

Evaluation of Possible Improvements: Several suggestions might be considered to better account for the coupling between pitch and elevation:

- **Suggestion A:**

Rewriting the elevation dynamics to be the following:

$$\ddot{e} = C_p V_s \cos(p) + C_e \cos(e),$$

and form a nonlinear state space model $\dot{x} = f(x, u)$.

- **Suggestion B:** Add a penalty term such as

$$\sum_{k=1}^N (\dot{e}_k - \dot{e}_{k-1} - \Delta t C_p V_s \cos(p_k) - \Delta t C_e \cos(e_k))^2$$

to the objective function.

- **Suggestion C:** Adapt the state weighting matrix Q in the LQ controller based on the current pitch angle, for instance by scaling the elevation penalty term with $\cos(p)$.

I explain how I would implement each of these and the experimental results in the section below:)

Conclusion: Even though the model is decoupled, the controller manages to work primarily because the approximations hold near the operating point and the robust feedback mechanism compensates for unmodeled dynamics. Enhancements that account for the coupling may improve performance but need to be balanced against additional complexity and stability concerns.

Here are the results for task 10.4 without any of the optional tasks on top of it. The results were satisfactory and the hurdle task was completed as wished. However it did not perhaps look as smooth as I had hoped. It went over the hurdle but did not do it as swiftly as I would have imagined. What I am trying to say is that I tried tuning it but it did not not dodge the hurdle like I wanted it to and we can see the elevation trajectory was not followed perfectly.

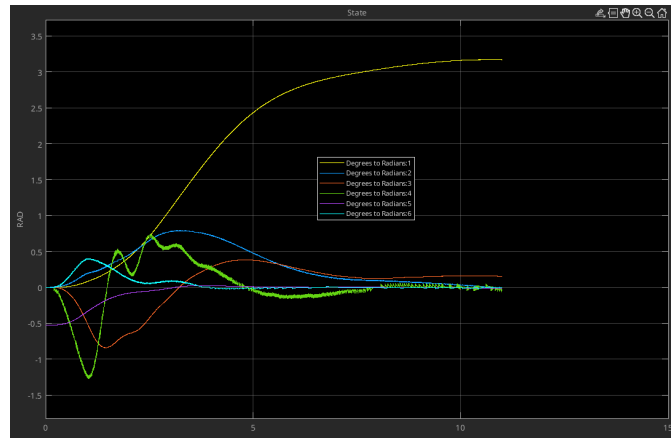


Figure 16: The state of task 10.4 vanilla.

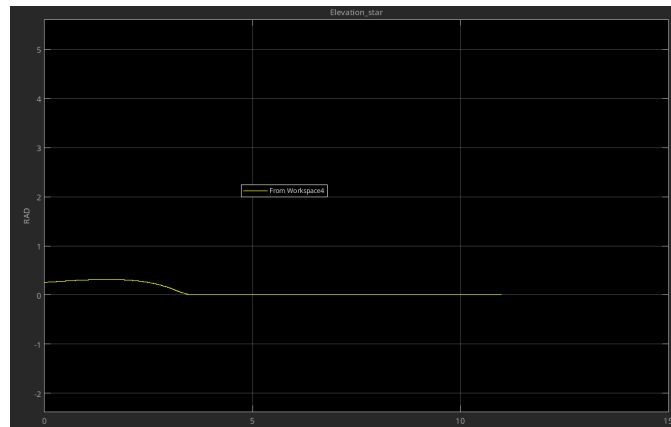


Figure 17: The elevation calculated trajectory of task 10.4 vanilla.

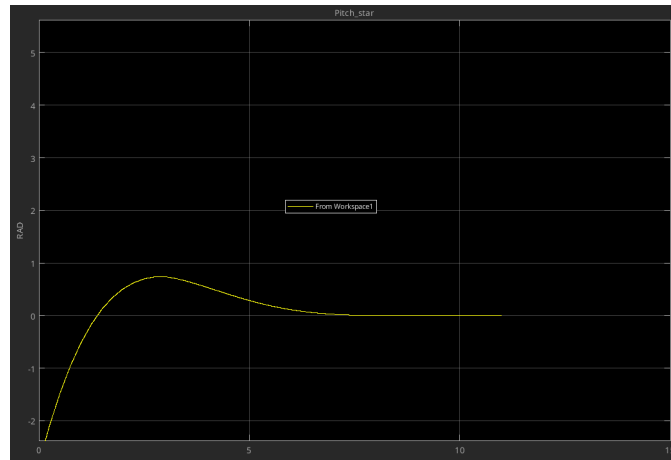


Figure 18: The pitch trajectory of task 10.4 vanilla.

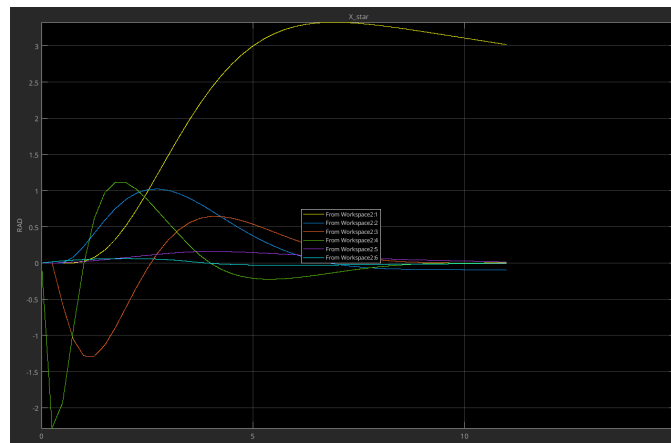


Figure 19: The vehicle state trajectory of task 10.4 vanilla.

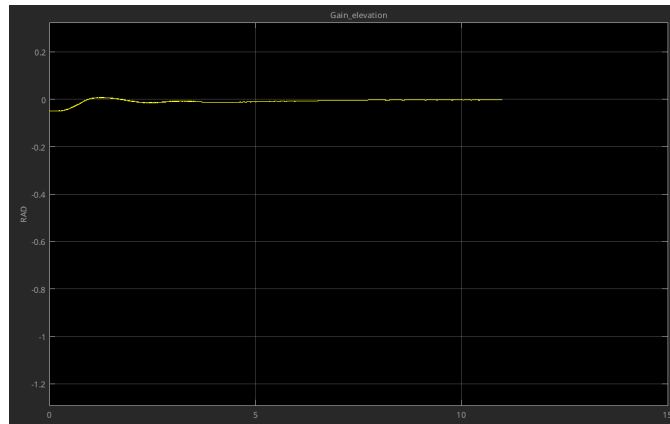


Figure 20: The elevation gain of task 10.4 vanilla.

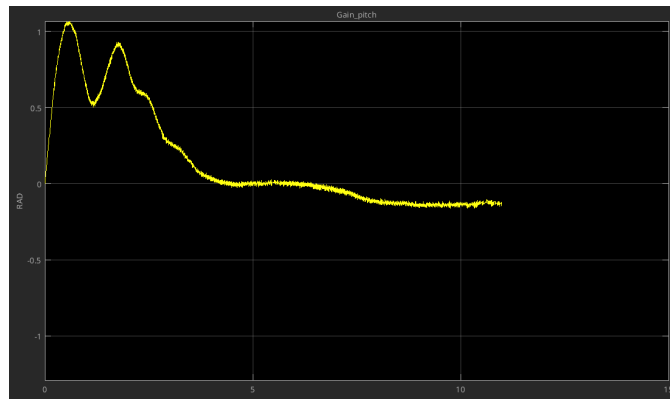


Figure 21: The pitch gain of task 10.4 vanilla.

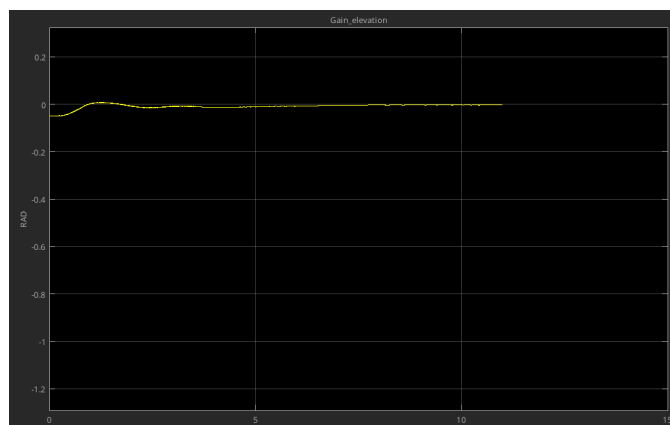


Figure 22: The elevation gain when implementing option B to model pitch elevation reference relationship.

3.5 MATLAB and Simulink

```
clear;
clc;
init09;

A_c = [0 1 0 0 0 0;
       0 0 -K_2 0 0 0;
       0 0 0 1 0 0;
       0 0 -K_1*K_pp -K_1*K_pd 0 0;
       0 0 0 0 0 1;
       0 0 0 0 -K_3*K_ep -K_3*K_ed];

B_c = [0 0;
       0 0;
       0 0;
       K_1*K_pp 0;
       0 0;
       0 K_3*K_ep];

dt = 0.25;
A = eye(6) + dt * A_c;
B = dt * B_c;

N = 40;
nx = 6;
nu = 2;

x0 = [0; 0; 0; 0; 0; 0];
lambda_f = pi;

alpha = 0.2;
beta = 20;
lambda_t = 2*pi/3;

q1 = 1;
q2 = 1;

z0 = zeros(N*nx + N*nu, 1);

options = optimoptions('fmincon','Display','iter','Algorithm','sqp');

cost_fn = @(z) cost_function(z, N, nx, nu, lambda_f, q1, q2);
con_fn = @(z) constraints(z, A, B, N, nx, nu, x0, alpha, beta, lambda_t);

[z_opt, fval] = fmincon(cost_fn, z0, [], [], [], [], [], [], con_fn, options);

x_opt = reshape(z_opt(1:nx*N), nx, N)';
u_opt = reshape(z_opt(nx*N+1:end), nu, N)';

time = (0:N-1)' * dt;

pc_timeseries = timeseries(u_opt(:,1), time);
ec_timeseries = timeseries(u_opt(:,2), time);
```

```

x_star_timeseries_dos = timeseries(x_opt, time);

save('opt_traj_elev_timeseries.mat', 'pc_timeseries', 'ec_timeseries', '
    x_star_timeseries_dos');

load('lqr_gain_dos.mat');
load('opt_traj_elev_timeseries.mat');
load('timeseries.mat');

%defining some functions here below since its cleaner imo
function J = cost_function(z, N, nx, nu, lambda_f, q1, q2)
    J = 0;
    for i = 1:N-1
        x_next = z(i*nx + 1 : (i+1)*nx);
        u_i = z(N*nx + (i-1)*nu + 1 : N*nx + i*nu);
        lambda_err = x_next(1) - lambda_f;
        J = J + lambda_err^2 + q1*u_i(1)^2 + q2*u_i(2)^2;
    end
end

function [c, ceq] = constraints(z, A, B, N, nx, nu, x0, alpha, beta,
    lambda_t)
    c = [];
    ceq = [];

    for i = 1:N-1
        x_i = z((i-1)*nx + 1 : i*nx);
        x_next = z(i*nx + 1 : (i+1)*nx);
        u_i = z(N*nx + (i-1)*nu + 1 : N*nx + i*nu);
        ceq = [ceq; x_next - (A*x_i + B*u_i)];
    end

    x1 = z(1:nx);
    ceq = [ceq; x1 - x0];

    for i = 1:N
        x_i = z((i-1)*nx + 1 : i*nx);
        lambda_k = x_i(1);
        e_k = x_i(5);
        e_limit = alpha * exp(-beta * (lambda_k - lambda_t)^2);
        c = [c; e_limit - e_k];
    end
end
end

```

Listing 3: Optimal control for 10.4

```

clear; clc;

init09;

A_c = [0 1 0 0 0 0;
       0 0 -K_2 0 0 0;
       0 0 0 1 0 0;

```

```

    0 0 -K_1*K_pp -K_1*K_pd 0 0;
    0 0 0 0 0 1;
    0 0 0 0 -K_3*K_ep -K_3*K_ed];

B_c = [0 0;
        0 0;
        0 0;
        K_1*K_pp 0;
        0 0;
        0 K_3*K_ep];

dt = 0.25;

A = eye(6) + dt * A_c;

B = dt * B_c;

Q = diag([3, 0.1, 2, 0.1, 1, 0.1])

R = diag([5, 5]);

[K, P, E] = dlqr(A, B, Q, R);

disp("K is ");
disp(K);

K_pitch = K(1,:);
K_elev = K(2,:);

K = reshape(K, 2, 6);
K_opt = zeros(1, 6);

save('lqr_gain_dos', 'K_pitch', 'K_elev', 'K_opt');
```

Listing 4: Finding K matrix for for 10.4

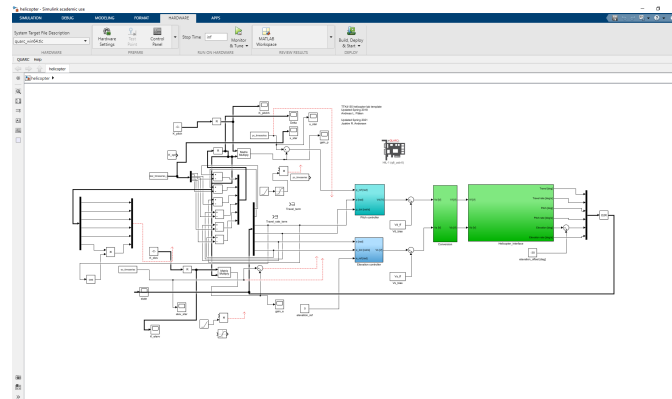


Figure 23: Simulink diagram of 10.4

3.6 Optional exercise

3.6.1 Option A

I derived the state space model again using the new dynamics of elevation, in order to model the true sinusoidal relationship between the pitch and elevation. This, however, is a non-linear, state space model. To solve this optimization we would need nonlinear optimization, or linearize multiple times and this could be both complex to make sure its done correctly, but also compute heavy.

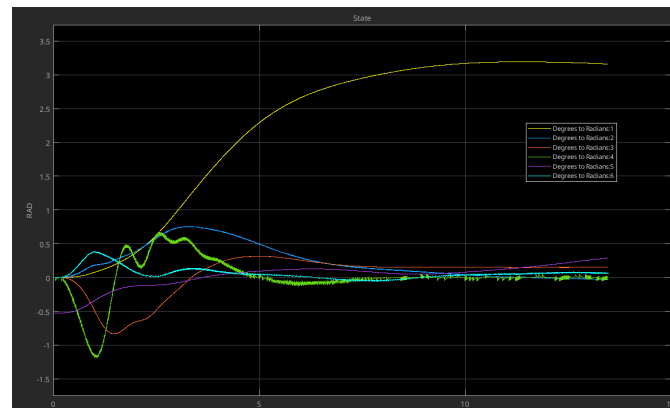


Figure 24: The state of implementing option A to model pitch elevation reference relationship.

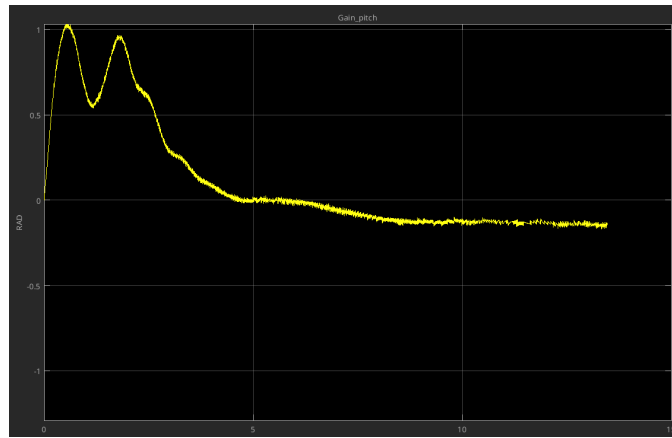


Figure 25: The pitch gain when implementing option A to model pitch elevation reference relationship.

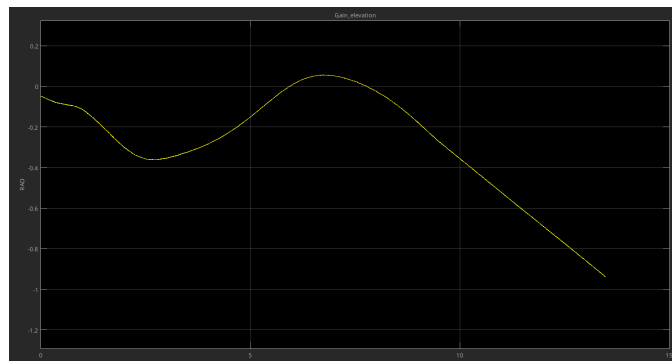


Figure 26: The elevation gain when implementing option A to model pitch elevation reference relationship.

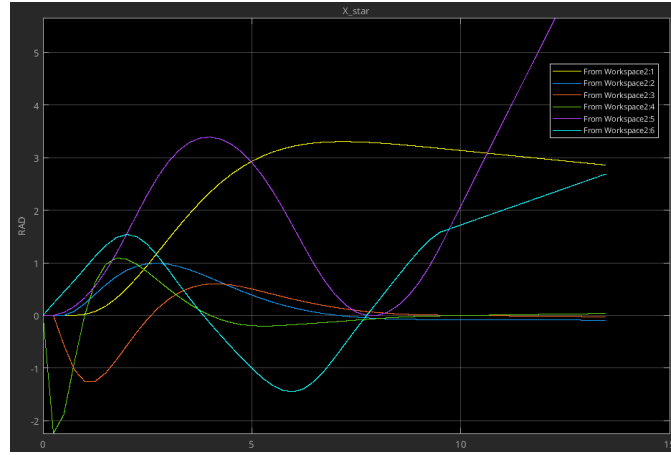


Figure 27: The optimal helicopter states when implementing option A to model pitch elevation reference relationship.

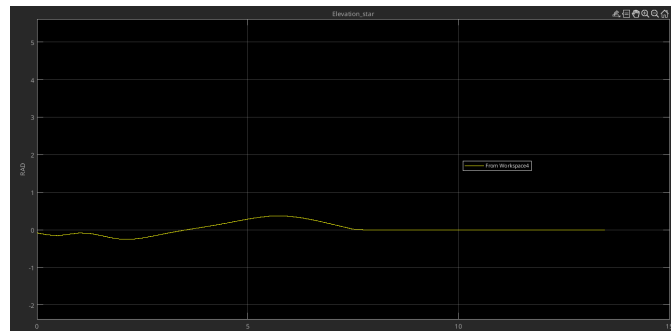


Figure 28: The elevation reference values when implementing option B to model pitch elevation reference relationship.

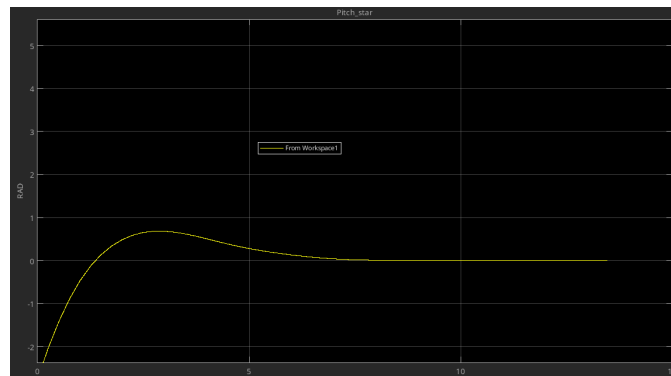


Figure 29: The pitch reference values when implementing option B to model pitch elevation reference relationship.

3.6.2 Option B

Here we would add a penalty term that enforces the dynamics described in the lab task. The issue with this option is that the optimization problem is now not quadratic, since we add a non-quadratic term. This would require nonlinear optimizers that are both computationally expensive and less reliable (i.e., they rely on a good first guess to produce good optimizations).

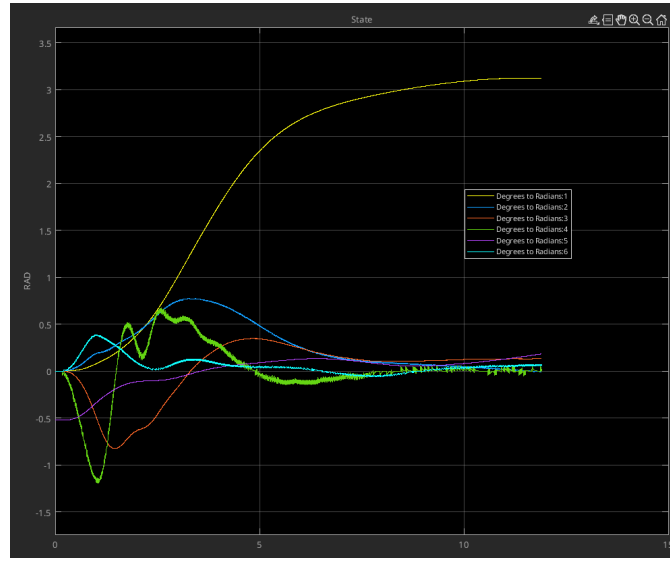


Figure 30: The state of implementing option B to model pitch elevation reference relationship.

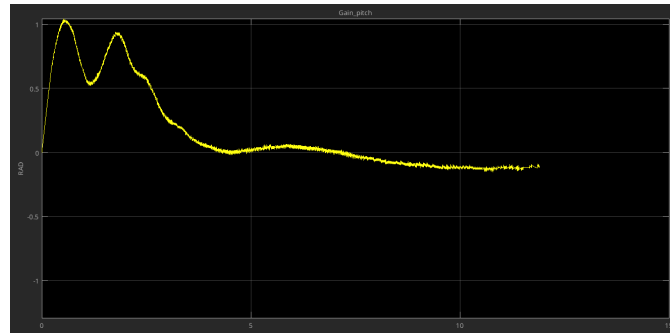


Figure 31: The pitch gain when implementing option B to model pitch elevation reference relationship.

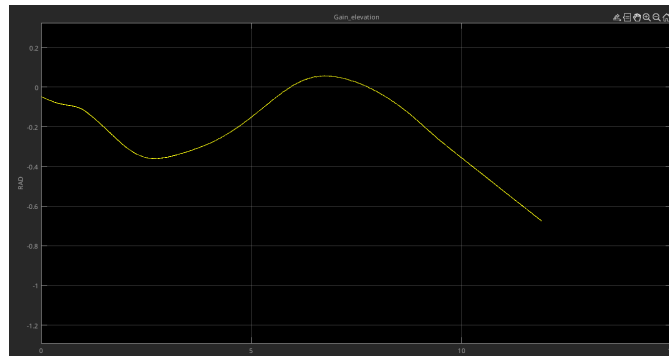


Figure 32: The elevation gain when implementing option B to model pitch elevation reference relationship.

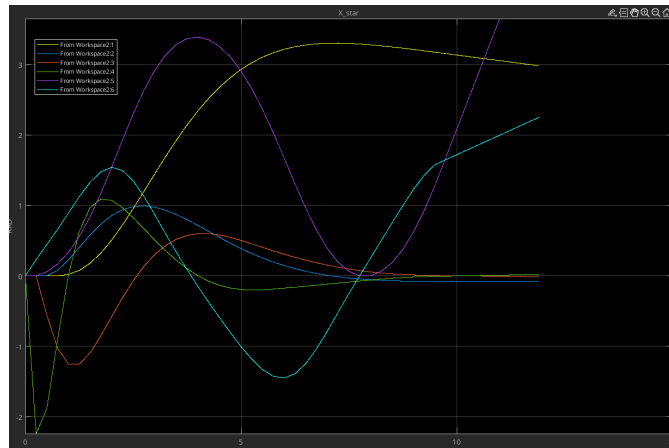


Figure 33: The optimal helicopter states when implementing option B to model pitch elevation reference relationship.

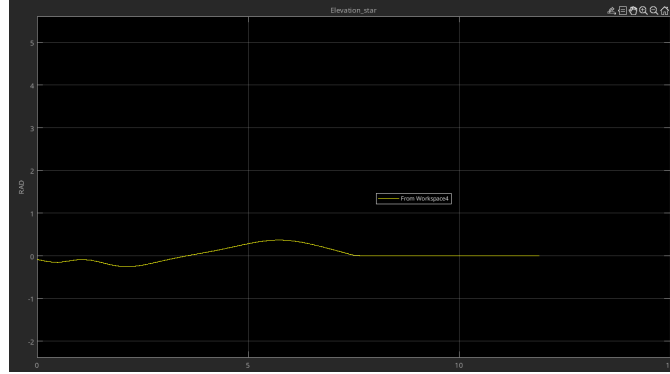


Figure 34: The elevation reference values when implementing option B to model pitch elevation reference relationship.

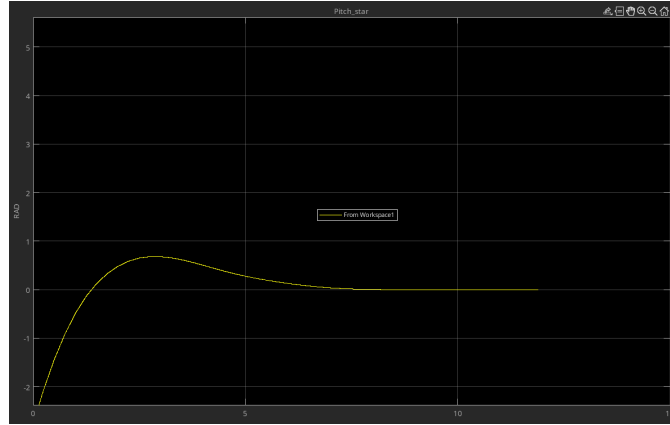


Figure 35: The pitch reference values when implementing option B to model pitch elevation reference relationship.

We can see the elevation gain just explodes and does not stabilize with time.

3.6.3 Option C

This option is to dynamically shift the LQR weight as to follow the pitch of the helicopter. Now this option is a bit more open-ended, which is why I liked it the most. It was first thought to be best to implement by changing the Q matrix as p changes, now this is what I did try initially. This proved a bit difficult and tested my matlab skills significantly, (even though I do have some proper training in Matlab previously I am not as fluent in it as other programming languages unfortunately). So my, you could say a bit more hacky, solution was to scale only the elevation feedback by the pitch of the helicopter, while running the experiment. The idea behind this is

that for one it would scale only the elevation change delta which is what we want, and two we are using the actually correct helicopter pitch to change it, and not the theoretically optimal pitch value. I would argue this is a more practical and functional solution, I do realize I am making this claim without implementing the other solution so I of course cannot state this with much conviction.

Here are the results for using this hacky fix, which you can see in the simulink diagram for problem 10.4.

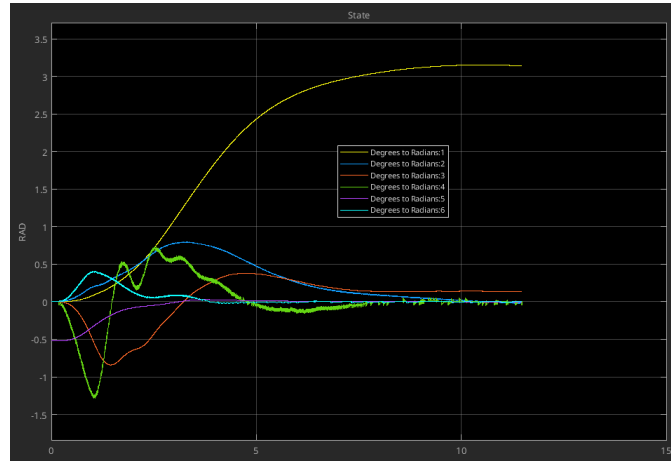


Figure 36: The state of implementing $\cos(p)$ scaled feedback on elevation reference.

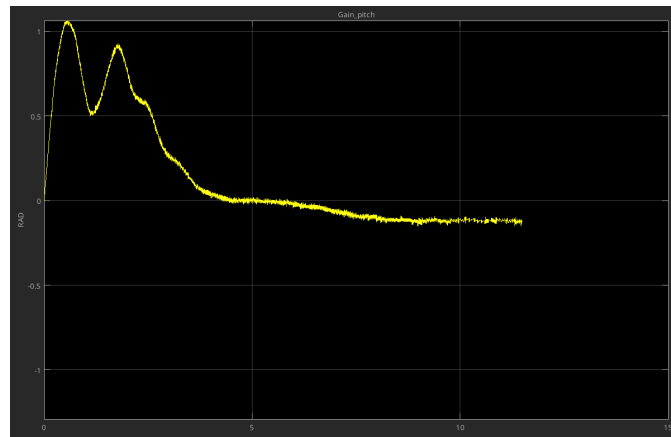


Figure 37: The pitch gain of implementing $\cos(p)$ scaled feedback on elevation reference.

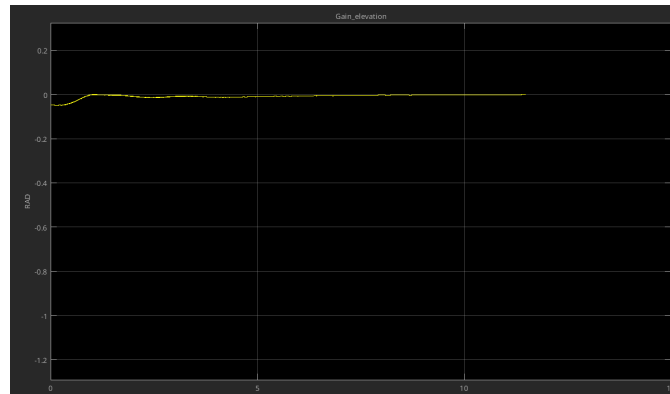


Figure 38: The elevation gain of implementing $\cos(p)$ scaled feedback on elevation reference.

Here the elevation gain stabilizes and we get the behavior we wanted. I've found in this lab that hacky, practical solutions are an important part of control engineering, at least for me who is not 100% solid on all of the theory, I might have made some grave errors in this report (if so, please let me know so I can learn). I found this lab one of the most fun ones I've had while at NTNU, so definitely encourage you guys to keep it for the following classes of students.